

Rapport de Projet

Reconnaissance de Nombres Binaires par OCR avec Réseau de Neurones MLP

Développement en Rust — Interface graphique Iced + Moteur OCR MLP

Introduction

Ce projet avait pour objectif de développer un système de reconnaissance optique de caractères (OCR) capable d'identifier des chiffres binaires dans des images, puis de convertir automatiquement la séquence reconnue en valeur décimale. L'intégralité du moteur de traitement a été écrite en **Rust**, associé à un réseau de neurones multicouche (MLP) pour la phase de classification. Ce projet m'a imposé de relever simultanément deux défis de taille : celui d'une architecture logicielle solide et celui d'un entraînement de modèle robuste, le tout dans un contexte personnel contraignant avec trois examens de rattrapage à préparer la même semaine. J'ai néanmoins refusé d'abandonner Rust, tirant au contraire les leçons de mes précédentes difficultés pour aborder ce projet avec une rigueur et une méthode bien supérieures.

Architecture générale du système

Côté client — Interface graphique (Iced)

L'interface utilisateur a été construite avec la bibliothèque **Iced** et constitue l'un des points forts du projet. Elle propose un visualiseur d'images paginé (boutons $\leftarrow$ $\rightarrow$), quatre actions principales (**Find**, **Load**, **Save**, **Solve**) et un champ de texte libre pour annoter les résultats. Les boutons déclenchent des messages qui transitent via un canal **mpsc** vers un thread worker dédié, garantissant que l'interface reste réactive pendant les traitements lourds. Les résultats de détection (bounding boxes) sont superposés en temps réel sur l'image sous forme de rectangles rouges grâce à un canvas Iced. La navigation gère correctement le dépassement d'index dans les deux sens et réinitialise l'état visuel à chaque changement d'image. Un soin particulier a été apporté au design : fond personnalisé, icônes dédiées pour chaque bouton, typographie monospace pour la barre de statut.

Côté serveur — Moteur OCR et pipeline de traitement

Le moteur tourne dans un thread Rust interne qui reçoit les requêtes **Find / Load / Solve** et renvoie les réponses structurées à l'UI. Le pipeline de traitement s'articule en plusieurs étapes : chargement de l'image, binarisation et normalisation, découpage des caractères en régions d'intérêt, vectorisation des patches puis inférence via le MLP. La réponse **Find** retourne un vecteur de coordonnées (xmin, ymin, xmax, ymax) pour chaque chiffre détecté ; **Solve** renvoie la paire de valeurs numériques correspondant à l'interprétation décimale de la séquence binaire reconnue.

Réseau de neurones MLP — Défis et erreur critique

Conception et entraînement

Le cœur du système repose sur un perceptron multicouche entraîné à distinguer les classes 0 et 1. La base de données utilisée comptait **12 000 images d'entraînement** et **2 000 images de test**. La conception du modèle s'est appuyée sur l'expérience accumulée lors des projets précédents, ce qui m'a permis d'aborder plus sereinement la structure du réseau et la phase de forward pass.

Deux jours de blocage : loss, biais et taux d'apprentissage

La phase la plus éprouvante du projet a sans conteste été l'entraînement du réseau, qui m'a mobilisé **plus de deux jours entiers**. Le calcul du loss ne convergeait pas : j'observais des oscillations persistantes quelle que soit la valeur du taux d'apprentissage (lr) testée. J'ai procédé à une revue systématique — vérification des dimensions des tenseurs, contrôle du gradient, ajustement manuel des biais couche par couche — sans succès immédiat. La recherche du bon lr s'est transformée en un processus d'essais minutieux, chaque modification demandant un nouveau cycle d'entraînement complet pour en évaluer l'effet.

L'erreur sous-jacente : un problème de découpe d'image

Après deux jours de débogage du modèle lui-même, j'ai finalement identifié la véritable cause : **un problème de prétraitement des images de test**. Lors du découpage et du redimensionnement des caractères avant inférence, les chiffres 0 et 1 touchaient les bordures de l'image redimensionnée, sans marge autour d'eux. Or, les images d'entraînement étaient, elles, parfaitement centrées avec du fond visible tout autour. Cette divergence entre les données d'entraînement et les données de test — invisible à première vue — suffisait à tromper systématiquement le réseau, qui avait appris à reconnaître des formes centrées et non des formes bord-à-bord. Corriger cette seule étape de prétraitement a immédiatement stabilisé le loss et rendu les prédictions correctes. Cette expérience m'a enseigné de façon concrète que **la qualité et la cohérence du prétraitement priment sur l'architecture du modèle**.

Organisation du travail et progression personnelle

Ce projet a été mené dans un contexte particulièrement chargé, avec trois examens de rattrapage à préparer simultanément. Pourtant, j'ai fait le choix de ne pas lâcher Rust malgré les difficultés rencontrées lors des rattrapages précédents. Ce projet aura été l'occasion de démontrer — surtout à moi-même — que ma façon de travailler a profondément évolué.

Avant d'écrire la moindre ligne de code, j'ai rédigé un plan détaillé couvrant l'ensemble des fonctionnalités à implémenter et réparti les tâches sur les jours disponibles. J'ai tenu un journal de bord quotidien où je consignais les avancées, les pistes à explorer le lendemain et les problèmes en suspens. J'ai également installé un environnement **Kubuntu** dédié afin de m'assurer de disposer d'un contexte stable, proche d'un environnement de production, pour observer les résultats en conditions réelles. Enfin, j'ai pris la décision de couper les écrans non essentiels — eau, fruits, concentration totale — et de persévérer jusqu'au bout.

En regardant l'ensemble de mes projets Rust sur ce semestre, la progression est nette : au premier, je n'avais pas même réussi à compiler car je m'étais persuadé de bien faire sans suffisamment tester. Au deuxième, j'ai intégré de nouveaux outils et multiplié les tests unitaires. Pour ce troisième projet, j'ai adopté une véritable démarche de développement logiciel professionnel. La dimension créative n'a pas été négligée pour autant : l'interface graphique, le design des boutons et l'attention portée à l'expérience utilisateur témoignent d'un investissement qui dépasse le simple exercice technique.

Conclusion

Les deux points forts de ce projet sont **une interface graphique soignée et un réseau de neurones fonctionnel**. L'erreur de découpe d'image, bien que coûteuse en temps, s'est révélée

être la leçon la plus précieuse du projet : elle illustre concrètement l'importance de la cohérence entre données d'entraînement et données de production. Sur le plan personnel, ce projet marque une étape de maturité dans mon rapport au développement : planification rigoureuse, persévérance face aux bugs profonds et capacité à remettre en question des hypothèses de départ solides en apparence. Ces acquis constitueront une base solide pour les futurs projets d'intelligence artificielle et de développement système.